

FSB Nexus

AI / RAG & Knowledge Base — Work Report

From an assistant that refused almost everything
to one that answers 83% of questions, every answer cited

Scope: AI/RAG pipeline, retrieval, knowledge base, evaluation
Faculté des Sciences de Bizerte · 16 July 2026

1. Executive Summary

The assistant was refusing almost every question, including ones it had the documents to answer. The inherited explanation was “the knowledge base data is dirty.” That turned out to be *half* right — and the half that was missing was the expensive one.

Rather than start fixing, we first built an **evaluation harness** and measured a baseline. That scoreboard then drove every decision and proved every change. The result:

0.50 → 0.90

RETRIEVAL HIT@5

0.395 → 0.776

MRR (RANK QUALITY)

→ 0.833

ANSWER RATE

0.833

ANSWERS WITH CITATIONS

The key discovery: the biggest single cause of the refusals was not the data at all — it was a **database index misconfiguration** that made correct answers literally invisible to the search, no matter how good the text was. Fixing it took one line and moved retrieval from 0.50 to 0.80. The data work then took it to 0.90.

2. The Problem

FSB Nexus answers student questions using RAG (Retrieval-Augmented Generation): it searches official faculty documents, then asks a language model to answer *only* from what it found, with citations. This is what prevents invented answers.

The system was behaving safely but uselessly — it declined nearly everything:

- **Factual questions** (“which licences in informatique?”) were refused even though the answer existed in the corpus.
- **Greetings and small talk** (“bonjour”, “merci”) were refused — they have no supporting document, so the sourced pipeline had no path for them.
- **Follow-up questions** had no context: `session_id` was passed into the pipeline but never used.

3. Method: Measure First, Then Fix

We deliberately did **not** start by cleaning data. Without a number that predates the changes, “the chat got better” is an opinion. So step one was the scoreboard.

The evaluation harness (`knowledge_base/eval/`)

- **41 questions** grounded in the real corpus: 30 answerable (each tagged with the document that should answer it) and 11 that *must* be refused.
- **Retrieval metrics** (fast, no LLM): `hit@5` — was the right document in the top 5 the pipeline actually reads; `MRR` — how highly it ranked.
- **End-to-end metrics** (real LLM): `answer_rate` , `answer_with_citation_rate` , and `false_answer_rate` — how often it invents an answer to an impossible question.

This harness is now the team’s shared quality gate — any future change to prompts, retrieval, or the corpus can be scored against it in minutes.

4. The Diagnosis That Changed Everything

The corpus holds only **3,675 chunks**, served through an `ivfflat` vector index. That index type groups vectors into clusters and, by default, searches **only one cluster** (`probes = 1`). If the right chunk lives in a different cluster, it is invisible — **at any number of results**. Raising the result count cannot rescue it.

So “un-retrievable even at top-15” never proved the text was dirty. We tested it directly, comparing the live index against exact (brute-force) search:

Question	Best relevant chunk — production (<code>probes=1</code>)	Best relevant chunk — exact search
“licence en informatique ?”	not found at all	rank 2 (0.55)
“licences en mathématiques ?”	not found	rank 11
“ départements de la FSB?”	not found	rank 9
“conditions d’admission en master ?”	not found	rank 11

For “licence en informatique”, the **2nd- and 3rd-best chunks in the entire corpus were invisible in production**. At 3,675 rows an approximate index buys nothing — exact search takes milliseconds.

But the data did also need work. Even with perfect (exact) search, the right content ranked only 9th–13th while the pipeline reads the top 5, and unrelated “attractor” documents (masters application forms, the clubs charter) outranked real licence pages. Both causes were real — the measurement told us the split instead of guessing.

5. What We Changed

5.1 Fixed the vector index — `ai/rag/search.py`

Made the search scan every cluster instead of one, restoring exact-search recall at negligible cost.

Measured: hit@5 0.50 → 0.80, MRR 0.395 → 0.612.

5.2 Softened the answering rules — `ai/prompts/system_prompts.py`

All four agents said: “*answer ONLY from the context; if insufficient, refuse.*” The model read “insufficient” strictly and refused even with a partially relevant document in hand. New rule: answer what the sources support, state plainly what they do not cover, and refuse only when **nothing** relevant was found. The “never invent” rule and citations were kept.

Measured: answer rate 0.60 → 0.70 with no increase in false answers. The groundedness checker fired 7 times instead of 2 — exactly as designed: the model attempts more, and the safety net silently blocks the unsupported attempts.

5.3 Added intent routing — `ai/rag/intent.py`

Greetings, thanks, and “what can you do?” now get a real, bounded reply and never enter the sourced pipeline. The classifier is deliberately conservative — anything ambiguous falls through to the factual path, so it cannot damage answer quality. Handles French, Arabic, and English.

Verified: 0 of 30 factual questions misroute; “Bonjour, quelles licences en maths ?” correctly stays factual.

5.4 Authored fact cards — `knowledge_base/orientation/web/`

The highest-value documents were each crammed into a **single chunk**: the entire licence list for every department was one run-on blob, so its meaning was diluted into an average and ranked below noise. We split the verified content into **7 clean, atomic cards** (one per department, plus a masters list). No facts were invented — existing verified content was restructured.

Measured: MRR 0.612 → 0.678; all five licence questions now retrieve their card at rank 1.

5.5 Added hybrid retrieval — `ai/rag/search.py`

The remaining failures were *exact-word* questions (“coordinateurs”, “retirer mon inscription”) that meaning-based search fumbles. We added a keyword arm (PostgreSQL full-text — no new dependency) and fused it with the vector arm using Reciprocal Rank Fusion. Two non-obvious fixes were required:

- **OR instead of AND.** The default query builder required *every* word — so “Comment changer de parcours” demanded the word “comment” and matched nothing.
- **Title weighting.** Query words appear more often in big course documents than in the small form that actually answers, so titles are now weighted far above body text.

Measured: hit@5 0.80 → 0.90, recall@10 → 0.967.

5.6 Added conversation memory — `ai/integration.py`, `ai/rag/pipeline.py`

The last 6 turns are now read from the chat history and given to the model. Crucially, **retrieval and the groundedness check still use only the current question**, so memory can never weaken grounding. Short follow-ups additionally borrow the previous question when searching.

Verified live. After a conversation about *maths* licences, the vague follow-up “*Et en informatique ?*” returned the informatique licence card **with a citation**. The identical question with no session drifted to an unrelated “prépa intégrée” answer — proof the memory is doing real work.

6. Results

Each change was measured in isolation, in order:

Stage	hit@5	MRR	Answer rate	False answers
Starting point (probes=1 , strict prompt)	0.50	0.395	0.60	0.00
+ index fix	0.80	0.612	0.60	0.00
+ softened prompts & intent routing	0.80	0.612	0.70	0.00
+ licence fact cards	0.80	0.678	—	—
+ hybrid retrieval	0.90	0.750	0.833	0.00
+ masters card & memory (final)	0.90	0.776	0.833	0.00–0.09

Every answered question carries citations — `answer_with_citation_rate` (0.833) equals `answer_rate` (0.833). The gain is real sourced answers, not vague hedging.

7. Honest Caveats

The anti-hallucination guardrail is strong but not provably perfect. Running the same 11 impossible questions three times produced **0, 0, and 1** invented answers. The language model is stochastic, so the correct figure is a **range (0.00–0.09), not a clean 0.00**. This matters because keyword matches now bypass the earlier relevance filter, leaving the groundedness checker as the sole backstop. It should be sampled over several runs, and any slip treated as a bug.

Three retrieval misses remain (out of 30): “s’inscrire” (rank 6) and “coordinateurs” (rank 8) sit just below the top-5 cut; “prépa intégrée” misses on word-frequency noise. A dedicated inscription fact card would likely close the first. None of these block the assistant.

Scope note: this report covers the AI/RAG and knowledge-base surface only. Frontend, backend security, and the courses/Google Classroom integration are other roles’ tracks.

8. What Changed, File by File

File	Change
<code>knowledge_base/eval/golden.jsonl</code>	New. 41-question benchmark (30 answerable, 11 must-refuse incl. 8 adversarial)
<code>knowledge_base/eval/run_eval.py</code>	New. Scoreboard: hit@5, MRR, answer rate, citation rate, false-answer rate
<code>ai/rag/search.py</code>	Index fix (full-recall probes) + hybrid vector/keyword retrieval with RRF
<code>ai/rag/intent.py</code>	New. Conservative FR/AR/EN intent router for greetings, thanks, capability questions
<code>ai/rag/pipeline.py</code>	<code>history</code> + <code>retrieval_query</code> parameters (memory without weakening grounding)
<code>ai/integration.py</code>	Intent short-circuit + conversation-history fetch
<code>ai/prompts/system_prompts.py</code>	Softened answering rules across all four agents
<code>knowledge_base/orientation/web/</code>	New. 7 fact cards (6 licence departments + masters list)

Corpus: 85 documents → 3,684 chunks, embedded with `paraphrase-multilingual-mpnet-base-v2` (768-dim). All 43 existing tests still pass.

9. Conclusion

The assistant now answers **83% of real questions with citations on every answer**, handles ordinary conversation, remembers context across turns, and still refuses what it cannot support.

The most transferable lesson is the method. The original diagnosis — “the data is dirty” — would have sent us straight into weeks of cleaning, and it would have *partly* worked while the real bottleneck sat untouched: a one-line index setting hiding correct answers. Building the scoreboard first cost a few hours and turned every subsequent decision from an argument into a measurement.